

# Software Architecture 2026

## Lecture 7, part 2/2

Architecture creation (ADD)

Architecture evaluation (ATAM)

Dimitri Van Landuyt

Laurens Sion

Wouter Joosen

# Architecture evaluation



# Architecture evaluation



## Architecture Trade-off Analysis Method (ATAM)

Analyzing architectural approaches

Identify risk and non-risk

Workshop-style

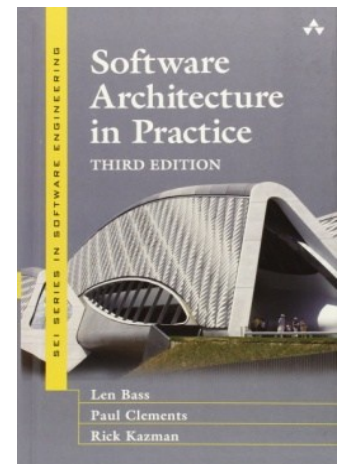
Involving key designers and stakeholders

Driven by ASRs/quality drivers

Focus on architectural trade-offs

# Architecture Trade-off Analysis Method (ATAM)

in a nutshell



# Architecture evaluation

## Different perspectives on evaluation

- › By the **architect** within the design process
  - ›› Every time a key decision is made, competing alternatives should be evaluated
- › By **peers** within the design process
  1. Reviewers determine the quality attribute scenarios to drive the review (e.g. security)
  2. The architect presents the portion of the architecture to be evaluated. For each scenario, the architect explains how the scenario is satisfied
  3. Potential problems are captured
- › Analysis by **outsiders** once the architecture has been designed
  - ›› Experts in specific areas
  - ›› Fresh perspective

# Architecture Trade-off Analysis Method (ATAM)

| Phase | Activity                    | Participants                                               | Typical Duration                                          |
|-------|-----------------------------|------------------------------------------------------------|-----------------------------------------------------------|
| 0     | Partnership and Preparation | Evaluation team leadership and key project decision makers | Proceeds informally as required, perhaps over a few weeks |
| 1     | Evaluation                  | Evaluation team and project decision makers                | 1-2 days followed by a hiatus of 1-3 weeks                |
| 2     | Evaluation (continued)      | Evaluation team, project decision maker, and stakeholders  | 2 days                                                    |
| 3     | Follow-up                   | Evaluation team and evaluation client                      | 1 week                                                    |

## Key participants:

- › Evaluation team [3-5 persons]
  - ›› Team Leader
  - ›› Evaluation Leader
  - ›› Scenario scribe
  - ›› Proceedings scribe
  - ›› Questioner
- › Project decision makers
- › Stakeholders

# Architecture Trade-off Analysis Method (ATAM)

Evaluation phase (phases 1 and 2)

## Phase 1

- › Step 1: Present the ATAM
- › Step 2: Present business drivers
- › Step 3: Present the architecture
- › Step 4: Identify architectural approaches
- › Step 5: Generate Quality Attribute Utility Tree
- › Step 6: Analyse Architectural Approaches

## Phase 2

- › Step 7: Brainstorm and prioritize scenarios
- › Step 8: Analyse Architectural Approaches
- › Step 9: Present results

# Architecture Trade-off Analysis Method (ATAM)

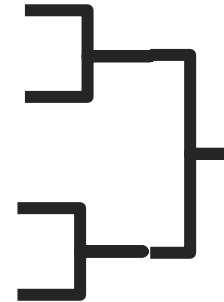
Evaluation phase (phases 1 and 2)

## Phase 1

- › Step 1: Present the ATAM
- › Step 2: Present business drivers
- › Step 3: Present the architecture
- › Step 4: Identify architectural approaches
- › Step 5: Generate Quality Attribute Utility Tree
- › Step 6: Analyse Architectural Approaches

## Phase 2

- › Step 7: Brainstorm and prioritize scenarios
- › Step 8: Analyse Architectural Approaches
- › Step 9: Present results



This is where the magic happens

# Architecture Trade-off Analysis Method (ATAM)

Evaluation phase (phases 1 and 2)

Phase 1

- › Step 1: Present the ATAM
- › Step 2: Present business drivers
- › Step 3: Present the architecture
- › Step 4: Identify architectural approaches
- › Step 5: Generate Quality Attribute Utility Tree
- › Step 6: Analyse Architectural Approaches

Phase 2

- › Step 7: Brainstorm and prioritize scenarios
- › Step 8: Analyse Architectural Approaches
- › Step 9: Present results

This is where the magic happens

Additional stakeholders are involved in phase 2

# Architecture Trade-off Analysis Method (ATAM)

Outputs/inputs of steps 1-5

Secondary outputs (*have to be created when they don't exist yet*):

1. Concise presentation of the architecture
2. Articulation of the business goals
3. Prioritized quality attribute scenarios

“Architecture description”

ASRs and  
Quality Attribute Utility Tree

# Architecture Trade-off Analysis Method (ATAM)

## › Step 4: Identify Architectural Approaches

- › Recognize tactics, patterns, reference architectures, reference approaches in the architecture and catalog them

## › Step 5: Generate Quality Attribute Utility Tree

- › Starting from high-level business goals
- › You know this step - the main difference is the ***architectural impact*** which now is much more tangible because it has materialized

## › Step 6: Analyze Architectural Approaches

- › Identify and catalog **sensitivity points, tradeoff points, risks, and non-risks.**

# Architectural Sensitivity Points

**Architectural Sensitivity Point** = an **architectural decision** that affects the degree to which the responses and response measures will be met

# Architectural Sensitivity Points

## Examples:

- » The **amount of database instances** will affect the amount of transactions a database can process per second (**performance**)
- » Power of encryption (**security**) is sensitive to **number of bits of the key**
- » The **shard key** or criterium affects the extent to which the database can grow over time (**scalability**)

# Architectural Trade-off Points

**Architectural Trade-off Point** = a sensitivity point (i.e. also an **architectural decision**) that affects **more than one quality attribute simultaneously**

# Architectural Trade-off Points

## Examples

- » Machine learning platform for recommendation can produce better result by increasing the sample size which in turn requires more resources (+ **Accuracy**, - **Cost/Performance**)
- » Multi-factor authentication (+ **Security**, - **Usability**)
- » Deciding to prioritize requests for risk calculation above research queries (+ **Performance**, - **Usability**)
- » Active replication (+ **Availability**, - **Performance**)
- » Active replication with eventual consistency and load balancing (+ **Availability**, + **Performance**, + **Scalability**, - **Data Consistency**)
- » The shard key or shard criterion (**Scalability**, **Performance**, **Availability**)

# Architecture Trade-off Analysis Method (ATAM)

Outputs/inputs

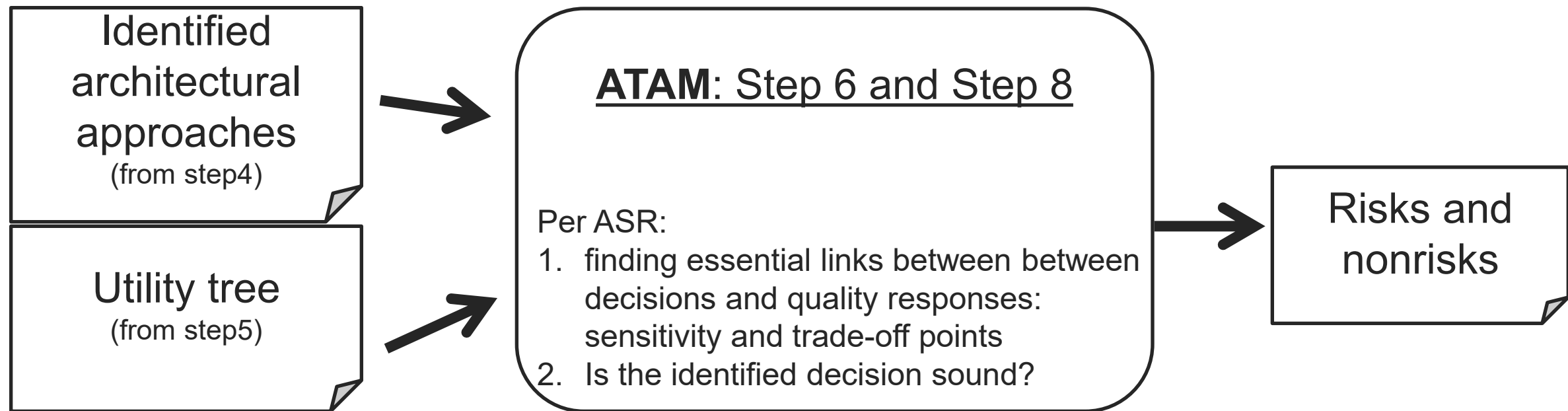
Along the way:

- › Mapping of **architectural decisions** to **quality requirements** (traceability)
- › The set of identified **sensitivity** and **tradeoff points**

Main outputs/end goals:

- › Identified **risks**: architectural decisions that were made and may lead to **undesirable consequences** in light of the stated quality attribute requirements
- › Identified **non-risks**: decisions that were made and, upon analysis, **are deemed sound**

# Architecture Trade-off Analysis Method (ATAM)



# Architecture Trade-off Analysis Method (ATAM)

› Output of the ATAM:

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                              |          |      |         |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------|------|---------|
| Scenario #: A12         |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Scenario: Detect and recover from HW failure of main switch. |          |      |         |
| Attribute(s)            | Availability                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                              |          |      |         |
| Environment             | Normal operations                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                              |          |      |         |
| Stimulus                | One of the CPUs fails                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                              |          |      |         |
| Response                | 0.999999 availability of switch                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                              |          |      |         |
| Architectural decisions |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Sensitivity                                                  | Tradeoff | Risk | Nonrisk |
| Backup CPU(s)           |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | S2                                                           |          | R8   |         |
| No backup data channel  |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | S3                                                           | T3       | R9   |         |
| Watchdog                |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | S4                                                           |          |      | N12     |
| Heartbeat               |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | S5                                                           |          |      | N13     |
| Failover routing        |                                                                                                                                                                                                                                                                                                                                                                                                                                                              | S6                                                           |          |      | N14     |
| Reasoning               | <p>Ensures no common mode failure by using different hardware and operating system (see Risk 8)</p> <p>Worst-case rollover is accomplished in 4 seconds as computing state takes that long at worst</p> <p>Guaranteed to detect failure within 2 seconds based on rates of heartbeat and watchdog</p> <p>Watchdog is simple and has proved reliable</p> <p>Availability requirement might be at risk due to lack of backup data channel ... (see Risk 9)</p> |                                                              |          |      |         |
| Architecture diagram    | <pre>graph LR; In(( )) --&gt; P[Primar CPU OS1]; In --&gt; B[Backup CPU with Watchdog OS2]; P -- "heartbeat 1 sec." --&gt; S[Switch CPU OS1]; B --&gt; S; S --&gt; Out(( ))</pre>                                                                                                                                                                                                                                                                            |                                                              |          |      |         |

Let's ATAM<sup>light</sup> the initial architecture

# Architecture Trade-off Analysis Method (ATAM)

Evaluation phase (phases 1 and 2)

## Phase 1

- › Step 1: Present the ATAM: **DONE**
- › Step 2: Present business drivers: **DONE**
- › Step 3: Present the architecture
- › Step 4: Identify architectural approaches
- › Step 5: Generate Quality Attribute Utility Tree
- › Step 6: Analyse Architectural Approaches

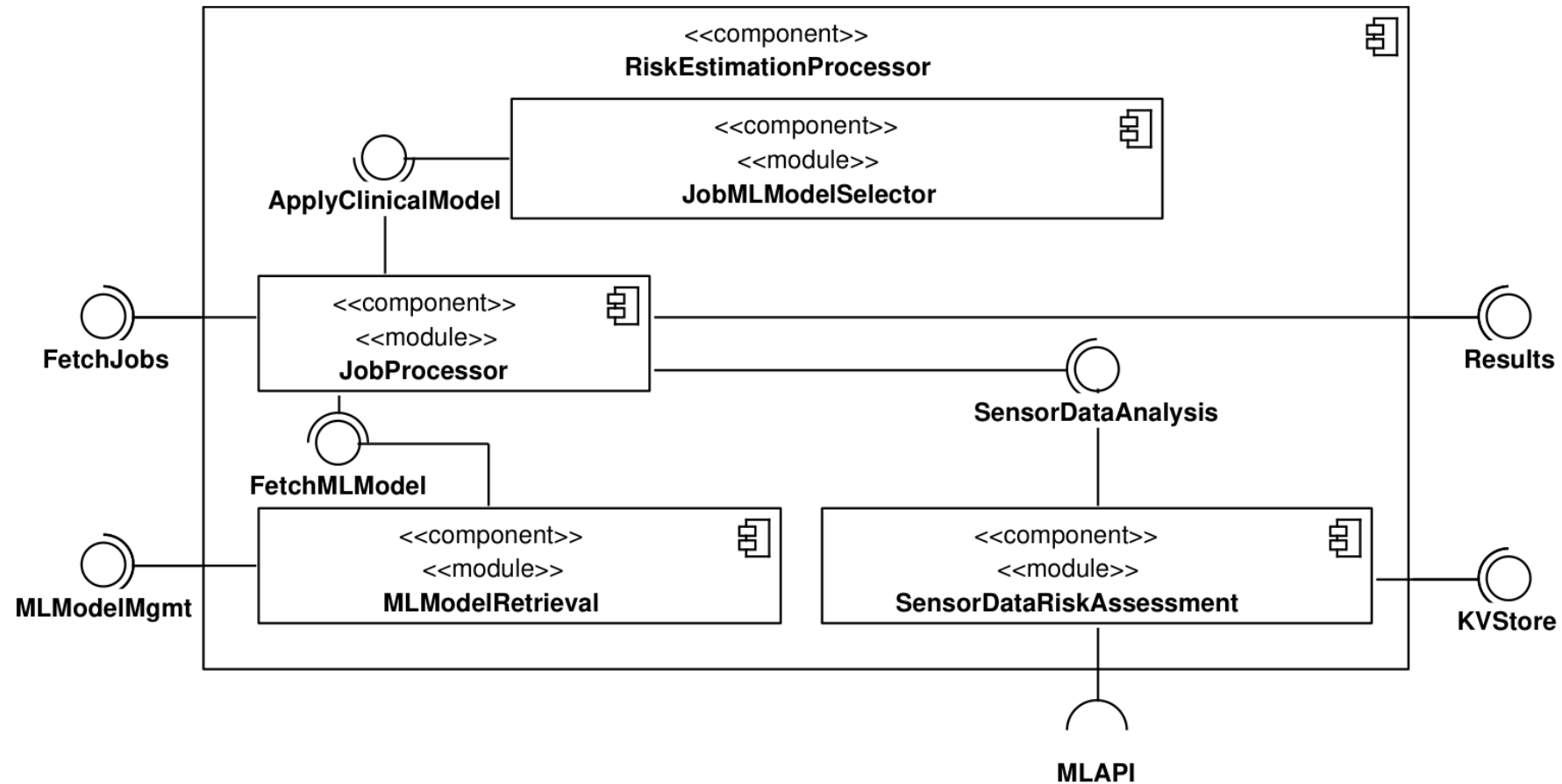
## Phase 2

- › Step 7: Brainstorm and prioritize scenarios
- › Step 8: Analyse Architectural Approaches
- › Step 9: Present results



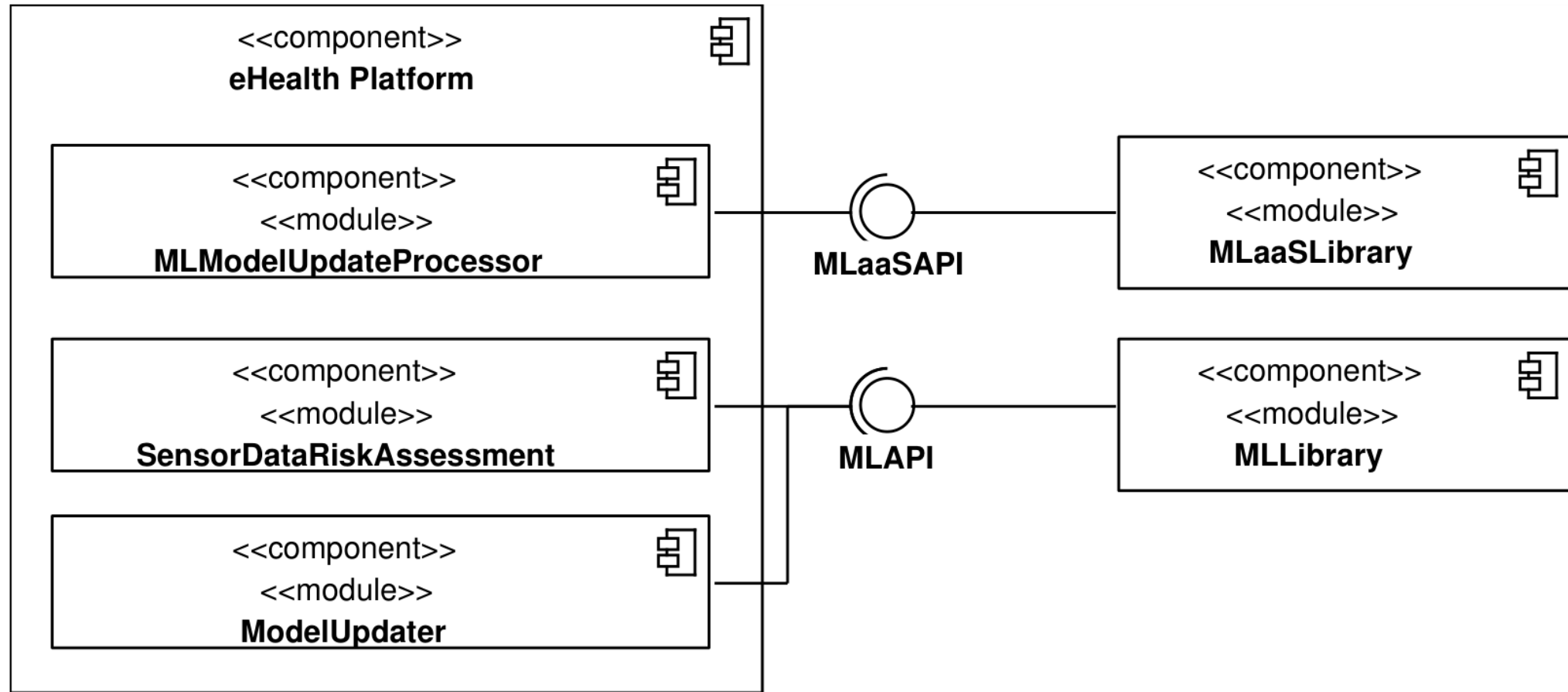
Step 3: Present the architecture

Step 4: Identify architectural approaches



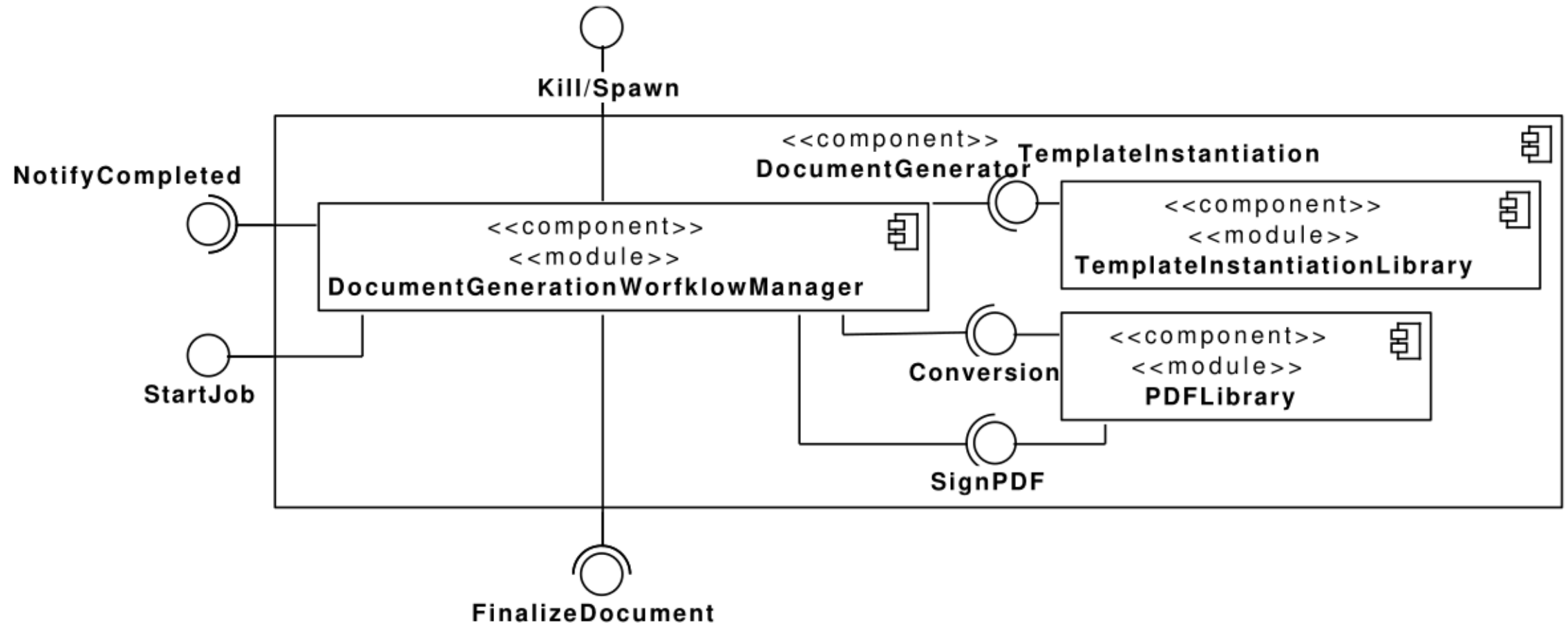
Step 3: Present the architecture

Step 4: Identify architectural approaches



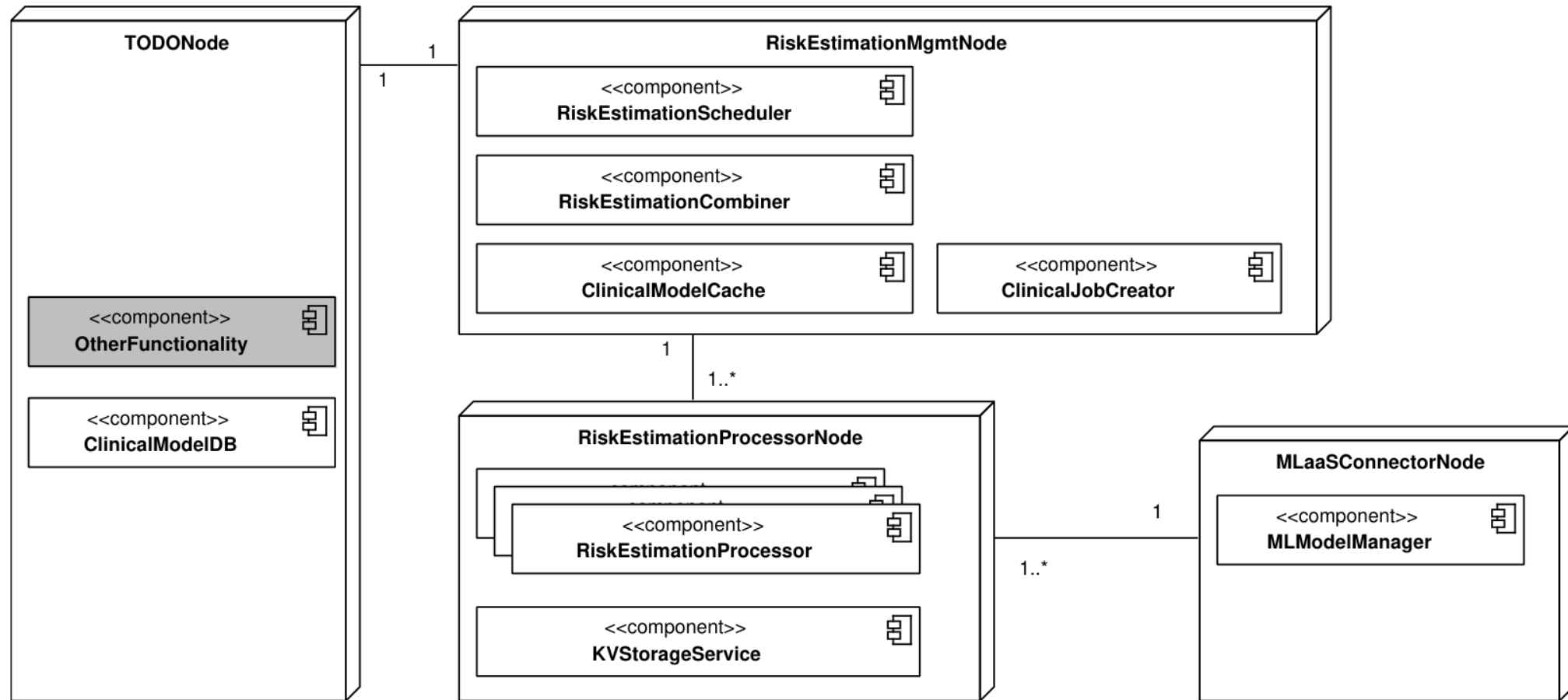
Step 3: Present the architecture

Step 4: Identify architectural approaches



Step 3: Present the architecture

Step 4: Identify architectural approaches



## Step 6: Analyze architectural approaches

- › Risks
- › Non-Risks

# 1. QAS Decisions

## 1.1 P2: Risk estimation by clinical models

### Key Decisions

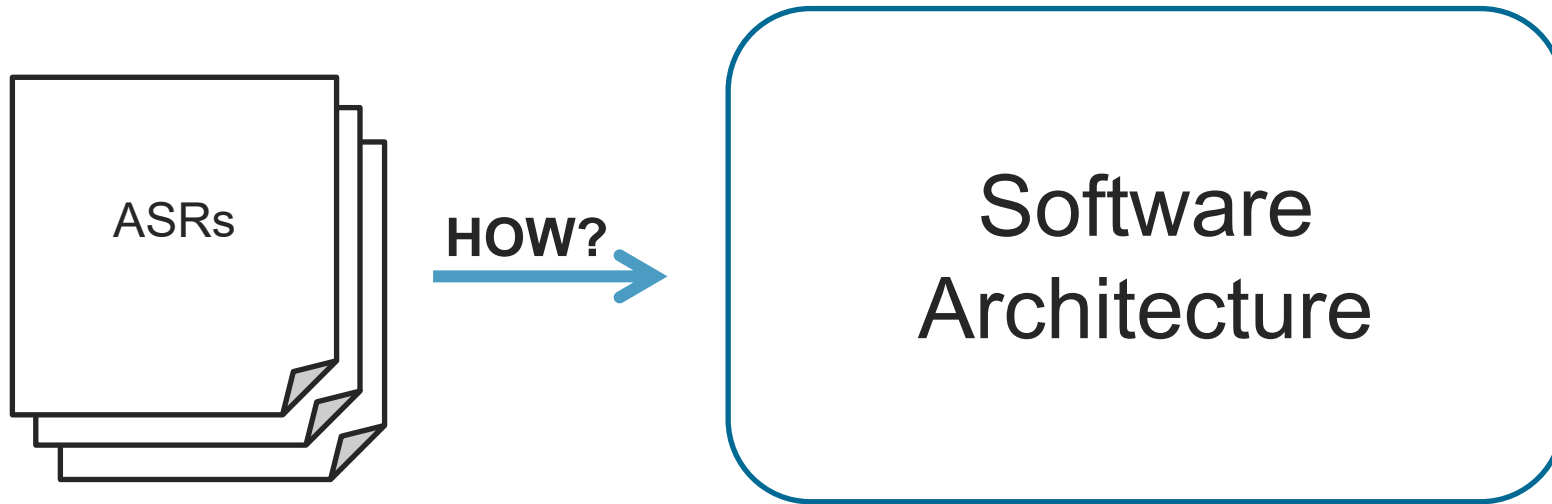
- (AD1) Parallelized and concurrent execution of multiple RiskEstimationProcessor instances, on multiple nodes and multiple times on each node
- (AD2) RiskEstimationScheduler queues new risk estimation **ClinicalModelJobs**, monitors the throughput and schedules them

### Architectural Decision 1: Parallel and concurrent risk estimation

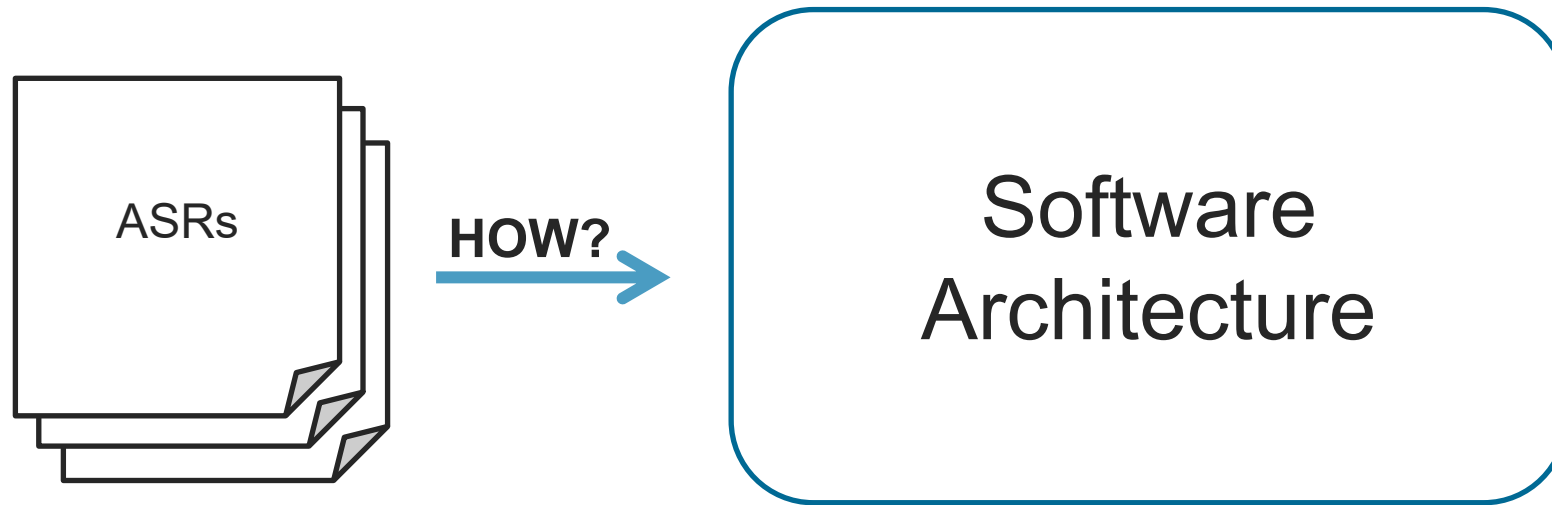
*P2* requires the execution of risk estimations to handle an increasing input load. To achieve this, multiple instances of the RiskEstimationProcessor are executed concurrently, allowing different risk estimations to be processed in parallel. Each processing node (RiskEstimationProcessorNode) asynchronously retrieves risk estimation jobs from a shared queue. These jobs can be treated as independent *commands* (an instance of the **Command** pattern), as risk estimations do not read or modify shared state. Consequently, no concurrency control mechanisms are required.



# Architecture creation



# Architecture creation



## Attribute-Driven Design (ADD)

- Top-down, step-wise refinement
- Hierarchical decomposition of the system
- Systematic/algorithmic
- Exploration of architectural alternatives
- Driven by ASRs/quality drivers
- Focus on architectural trade-offs

# A simple pseudo-algorithm?

```
SA our_architecture = new SA();
```

```
for(ASR asr : asrs)
```

```
    our_architecture.satisfy(asr);
```

# A simple pseudo-algorithm?

```
SA our_architecture = new SA();
```

```
for(ASR asr : asrs)
```

```
    our_architecture.satisfy(asr);
```



Which order? Does it matter?  
One by one? Grouping of architectural drivers?

# Attribute-Driven Design (ADD)

in a nutshell

# Attribute-driven design (ADD)

## 1. Not all requirements matter

- » **Subset** of the requirements that **shape the architecture** (both functional and non-functional): **ASRs**

## 2. Not all requirements are equal (utility tree)

- » Problem reduced to satisfying the most important ones (i.e., the **architectural drivers**)
- » More precisely: the less important requirements are satisfied **within the constraints of the more important**

**ADD philosophy:** **quality attributes** shape the architecture  
(drive the decomposition of the architecture)

- » Functionality is assigned as a “consequence”

# Attribute-driven design (ADD)

- › An **iterative decomposition** process where, at each stage, **tactics** and **architectural patterns** are chosen to satisfy a set of **quality attributes** and then **functionality** and **residual drivers** are allocated to the resulting structure
- › **Outcome:** First several levels of a **hierarchical decomposition**, where the system is decomposed into its constituting elements and constraints are imposed on how they interact

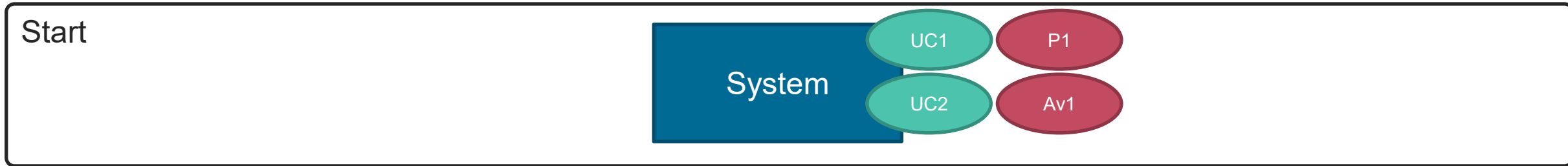
# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**

And repeat until done

# Attribute-driven design (ADD): high level overview

Ad-hoc notation!



# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**

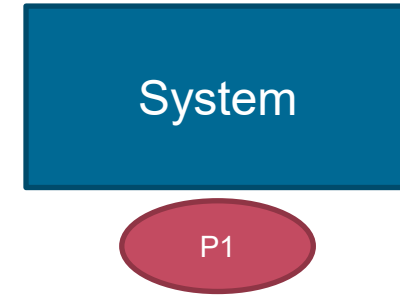


And repeat until done

# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**

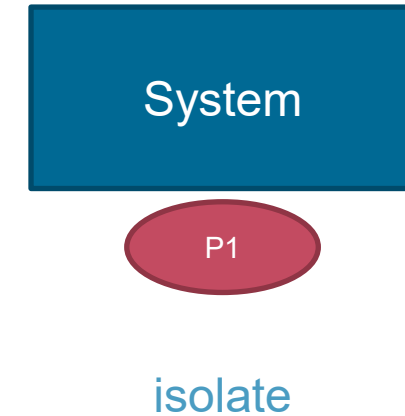
And repeat until done



# Attribute-driven design (ADD)

- a. Pick an element that needs to be decomposed
- b. Identify drivers (important qualities first)
- c. Select a solution
- d. Instantiate solution
- e. Verify and refine

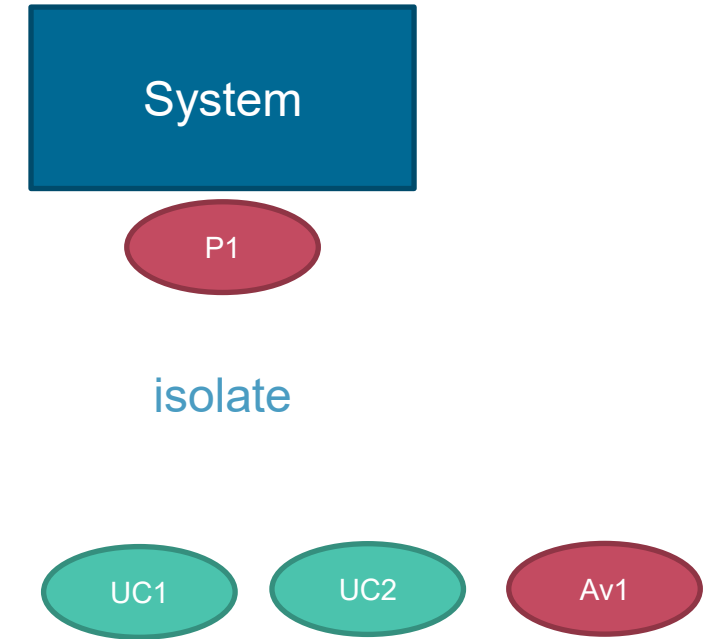
And repeat until done



# Attribute-driven design (ADD)

- a. Pick an element that needs to be decomposed
- b. Identify drivers (important qualities first)
- c. Select a solution
- d. Instantiate solution
- e. Verify and refine

And repeat until done



# Attribute-driven design (ADD): high level overview

Ad-hoc notation!

Start



## ADD run 1

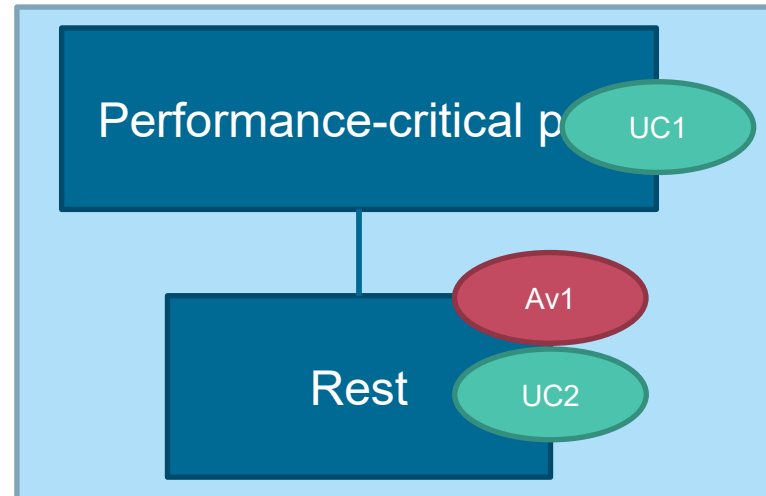
Decompose 'System'

Driven by



Chosen solution:

isolate



# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**



And repeat until done

# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**

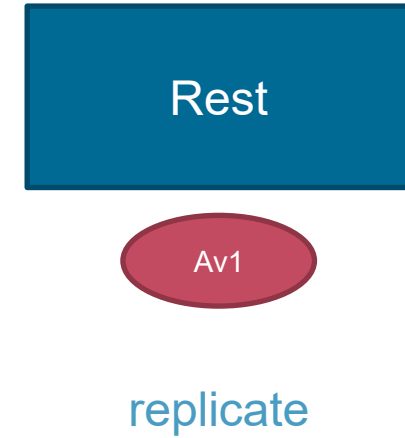
And repeat until done



# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**

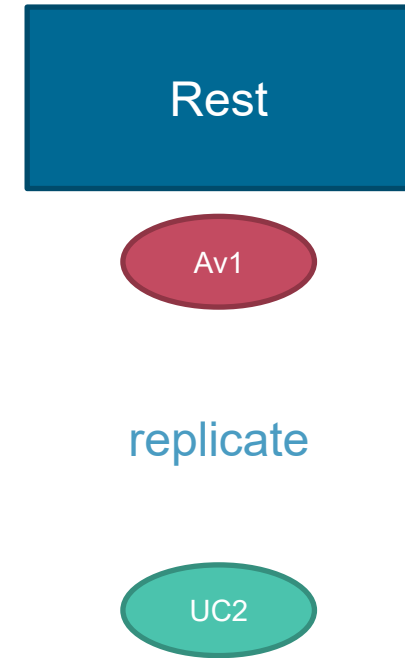
And repeat until done



# Attribute-driven design (ADD)

- a. Pick an **element** that needs to be decomposed
- b. Identify **drivers** (important qualities first)
- c. Select a **solution**
- d. **Instantiate** solution
- e. **Verify** and **refine**

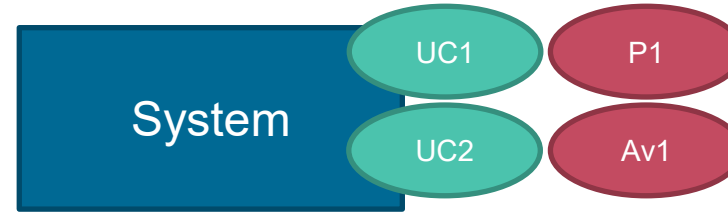
And repeat until done



# Attribute-driven design (ADD): high level overview

Ad-hoc notation!

Start



## ADD run 1

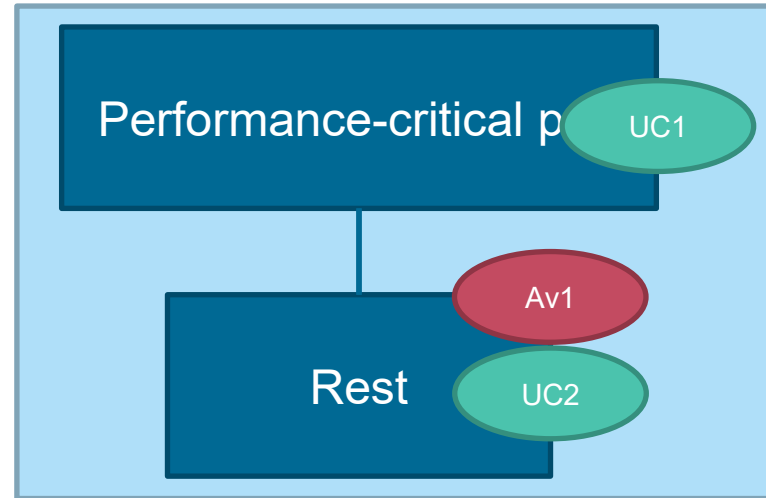
Decompose 'System'

Driven by



Chosen solution:

isolate



## ADD run 2

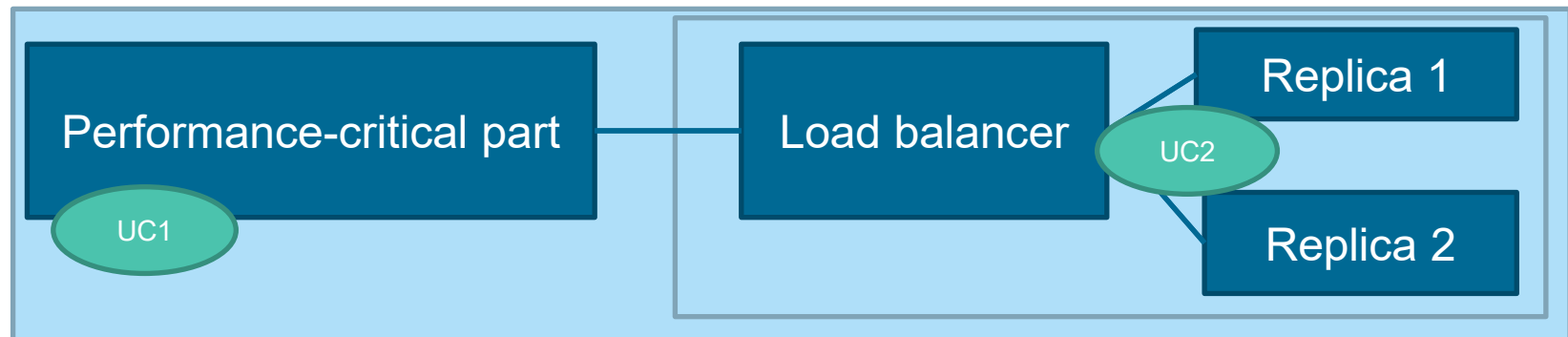
Decompose 'Rest'

Driven by



Chosen solution:

replicate



# This is an oversimplification

- › **Grouping** related architectural drivers
- › **Splitting** up different aspect of requirements drivers: declaring **residual requirements** (e.g., P1-Response2, M1-Response4, ...)
  - ›› E.g. dealing with unrelated responses in separate iterations

# This is an oversimplification

Quality requirements may not be **satisfiable** with the current decomposition

Two options:

1. **Reconsider** earlier decisions (backtracking)
  - » OK, but costly...
  - » Reason to invest in documenting alternatives considered!
2. investigate possible **trade-off relaxation of the requirement**
  - » decisions made earlier were in function of the more important requirement

This situation shows us a **trade-off point** in the architecture  
multiple qualities interact (pos/neg)



# Architectural trade-offs in ADD

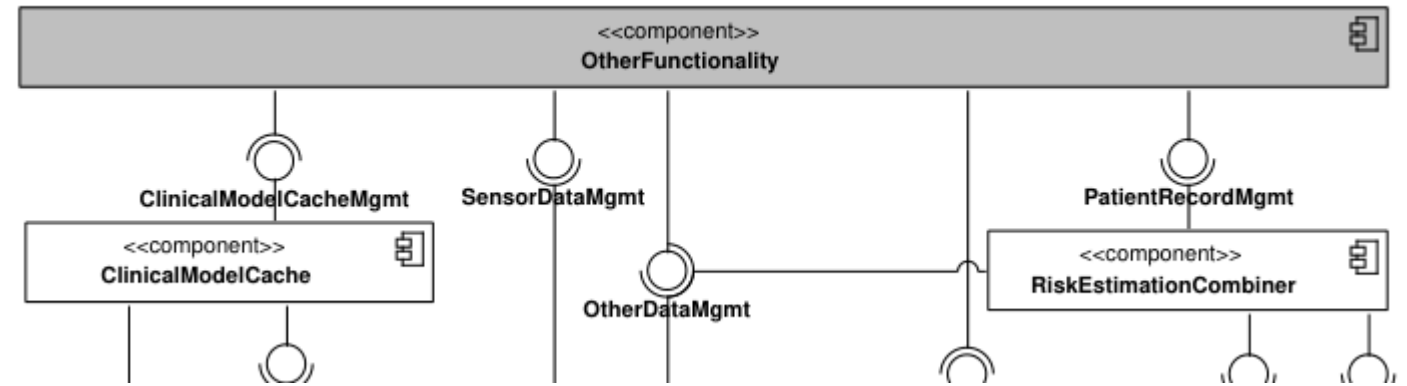
## 1. Through **sequential runs of ADD** :

- Important drivers first have **largest impact** – constraints are imposed upon the architecture through hierarchical decomposition
- Subsequent drivers should not undo or counteract these decisions

## 2. By picking **appropriate combinations of drivers to address in a single run**

# The initial architecture is the outcome of two ADD iterations

- › “OtherFunctionality” as a **placeholder**
  - ›› Coarse-grained grouping of functionality not yet addressed (no further decomposition into concrete architectural elements)
  - ›› Simply because it was not required for the first two iterations
  - ›› In that sense, the ADD process is **not yet complete**

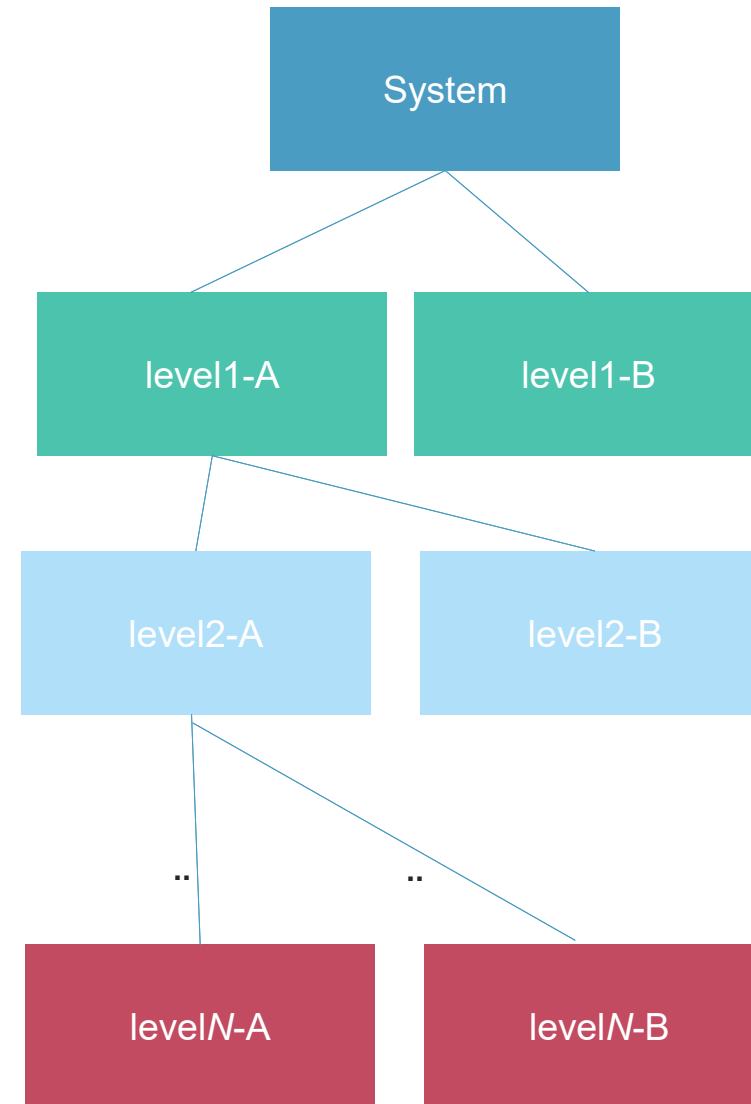


=> in the final architecture, the aim is to **not** have such residual structures anymore

- › In case NFRs are left -> more iterations of ADD
- › In case only functional requirements (use cases) are left -> make a rough functional decomposition (with degrees of freedom for the developer)

# ADD may increase nesting complexity

- › Because it is based on recursive decomposition of prior elements
  - › In the worst case: after  $n$  iterations of ADD, you may have  $n$  levels of tree depth
- › It's fine to “prune” this tree
  - › (decompose but also replace entirely the element you decompose)
- › Rule of thumb: 2-3 levels should suffice (think about granularity and revise when necessary)
  - › Otherwise, modeling complexity becomes prohibitive (remember: SAPPlugin forces consistency over all the levels!)



# Complementary methods



## Attribute-Driven Design (ADD)

Top-down, step-wise refinement  
 Hierarchical decomposition of the system  
 Systematic/algorithmic  
 Exploration of architectural alternatives

## Architecture Trade-off Analysis Method (ATAM)

Analyzing architectural approaches  
 Identify risk and non-risk  
 Workshop-style  
 Involving key designers and stakeholders

**Driven by ASRs/quality drivers**  
**Focus on architectural sensitivity and trade-offs**

# Complementary methods



## Attribute-Driven Design (ADD)

Top-down, step-wise refinement  
 Hierarchical decomposition of the system  
 Systematic/algorithmic  
 Exploration of architectural alternatives

## Architecture Trade-off Analysis Method (ATAM)

Analyzing architectural approaches  
 Identify risk and non-risk  
 Workshop-style  
 Involving key designers and stakeholders

**Driven by ASRs/quality drivers**  
**Focus on architectural trade-offs**

**Architectural utility**

# Software Architecture

Driven by **ASRs/quality drivers**  
Focus on **architectural trade-offs**

Architectural utility

This is how your result will be evaluated

### **How does your architecture fare in terms of utility?**

(assuming that you make no mistakes against system goals/domain knowledge, or against modeling and clarity in communication)

- Do you address the requirements in line with their intrinsic relevance and priority?
  - Are your architecture approaches well-reasoned, clear, and sound?
    - Are you aware of the sensitivity points?
  - Do you make informed trade-off decisions?

**Architectural utility**

# Organizing the work

## › Importance of drivers

- › Think beforehand in which order you will tackle them: this has an impact!
- › **Grouping**: these requirements **will interact**!
- › **Prioritize**: respect the intrinsic priorities

# Organizing the work

## Team

### › Your call entirely/your team can self-organise

1. Read carefully, try to identify interactions between drivers early on
2. Modeling: do not let it get in the way - try to avoid costly backtracking
3. Approach:
  - ADD can be an ***inspiration***: prioritize drivers, group drivers (or elements thereof) that belong together, solve iteratively through hierarchical decomposition
  - ADD can also be a ***strict methodology*** for you to follow
  - Adopt the ATAM mindset to critically self-evaluate your decisions: **risk** or **nonrisk**?

**Brainstorm - Discuss - Evaluate - Decide - Model**

***“Split up and bring together individual contributions”  
does not work well***

“The problems we face have an amazing ability to scale. Put five problems in a space, and they can all interact to make things 25 times worse. [..]

Put five problem solvers in a room, however, and it is very unlikely that we will get even five times the problem-solving capability.”

*Tom Munnecke*

# Definitions

- › Lecture 1: “*the bridge between requirements and design*”
- › Lecture 2: “*meeting or maximizing utility*”
- › Lecture 3: “*{elements, form, rationale}*” | “*the set of principal design decisions*” | “*the fundamental organization of a system, embodied in its components, their relationships [..], and the principles guiding its design and evolution*”
- › Lecture 4: a collection of *views*, [addressing *stakeholder concerns* according to well-defined *viewpoints*, further concretized in specific *models* (e.g. diagrams)], augmented with *rationale*
- › Lecture 5: “
- › Lecture 6: managing **architectural sensitivity and trade-off points** by
  - (i) laying down **constraints** which ensure that the important non-functional requirements will be satisfied (ADD), and (ii) are considered **nonrisk** (ATAM)